



Sanjeevni

Government Electronic Health Record System (GovEHR)

An all-in-one centralized platform for government hospitals to record, track, and manage patient health history with AI-assisted clinical decision support.

Smart India Hackathon (SIH) Project

Technology Stack: Next.js 14 · TypeScript · Tailwind CSS · PostgreSQL (Neon) · Gemini AI · AWS S3

August 2026

Table of Contents

1. Project Overview
2. Problem Statement & Motivation
3. System Architecture
4. Technology Stack
5. Main Features (6 Portals)
6. End-to-End Workflow
7. Database Design & Schema
8. AI Triage Engine
9. Treatment Compaction (USP)
10. OCR & Prescription Extraction
11. Multilingual Support
12. Identity Verification
13. API Endpoints
14. Setup & Deployment
15. Viva Preparation — Frequently Asked Questions

1. Project Overview

Sanjeevni (named after the mythical healing herb from the Ramayana) is a comprehensive, centralized Government Electronic Health Record (EHR) and Hospital Management System designed for government hospitals across India. It provides a unified platform where patients, doctors, lab technicians, and hospital kiosks can interact with a single patient health record throughout their lifetime.

Core Vision: Every citizen of India gets a single Unique Health ID (HID) that follows them across all government hospitals. All medical data — visits, prescriptions, lab results, self-reported vitals, surgical history, vaccinations — is chained as "clinical nodes" in a lifetime timeline. After each treatment completes, the data is *compacted* into a summary, keeping the active database lean while preserving full historical context.

Key Highlights

- **6 Role-Based Portals:** Patient Portal, Kiosk Mode, Lab Portal, Doctor Clinical View, Doctor Prescription Builder, Patient Intake Wizard
- **AI-Powered Triage:** Built-in clinical decision support engine that checks drug interactions, allergy cross-reactivity, polypharmacy risks, critical lab values, and vital sign abnormalities
- **Multilingual UI:** Supports English, Hindi, and Bengali through a DOM-level translator and i18n framework
- **Treatment Compaction:** Unique data management approach where completed treatment chains are compressed into summaries, maintaining a clean active database
- **Voice Input:** Kiosk mode supports voice-based patient registration with multilingual speech transcription
- **OCR Prescription Scanning:** Upload prescription images and extract medications using Google Gemini AI
- **Neon PostgreSQL:** Serverless cloud database with automatic schema creation and mock data seeding

2. Problem Statement & Motivation

The Problem

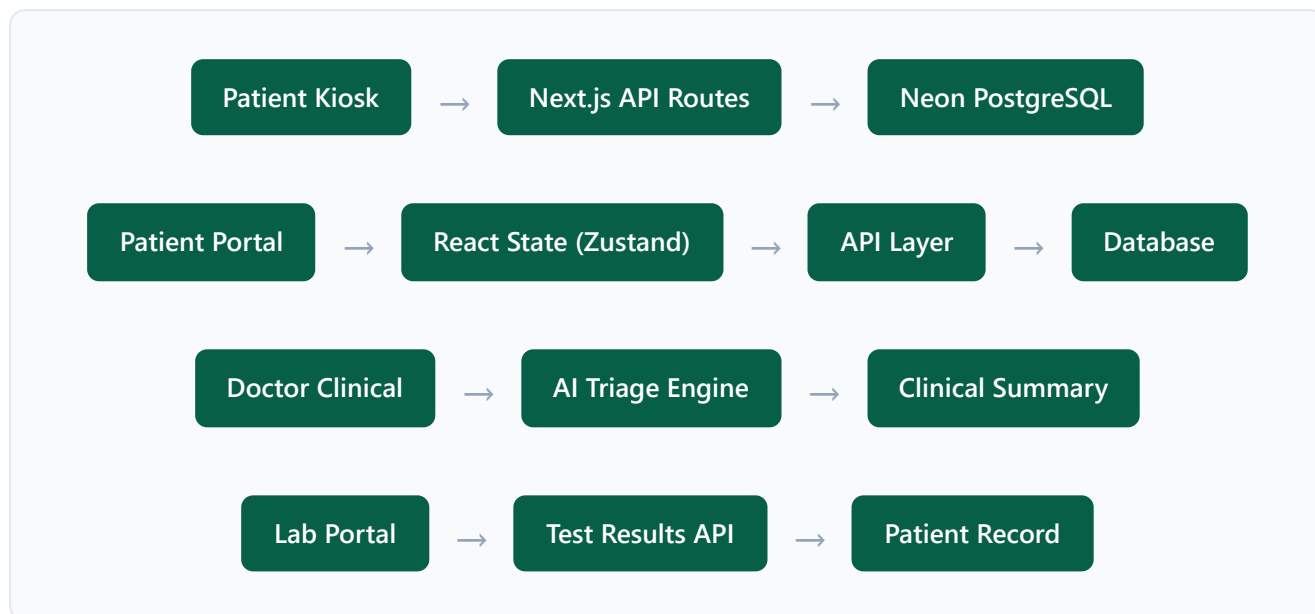
- Government hospitals in India still rely on **paper-based records** or fragmented digital systems
- Patient records are **siloed** within individual hospitals — there is no central repository
- Patients visiting multiple hospitals must **carry physical files** or repeat tests
- Doctors lack a **complete medical history** when treating patients from other hospitals
- Language barriers prevent effective communication in **multilingual India**
- No standardized system for **AI-assisted clinical decision support** in government hospitals

Our Solution

Sanjeevni creates a unified, lifetime health record for every patient with a Unique Health ID. The system captures data from multiple entry points (kiosk registration, patient self-report, doctor consultations, lab results, prescription scanning) and chains them into a coherent clinical timeline. An AI triage engine provides real-time clinical decision support to doctors, and treatment compaction ensures the system remains performant over decades of patient history.

3. System Architecture

High-Level Architecture



Frontend Architecture

- **Framework:** Next.js 14 (App Router with Server Components)
- **State Management:** Zustand (single global store for all application state)
- **UI Components:** Custom component library built on Radix UI primitives + Tailwind CSS
- **Routing:** Role-based view switching via Zustand store (not URL routes)
- **i18n:** Dual approach — React context for Kiosk translations + DOM-level tree walker for page-wide translation

Backend Architecture

- **API Routes:** 11 Next.js API route handlers under `/api/*`
- **Database:** Neon PostgreSQL with lazy initialization and automatic schema creation
- **Transactions:** PostgreSQL transaction wrappers for atomic multi-table operations
- **AI Integration:** Google Gemini API for OCR and medical summarization
- **Storage:** AWS S3-compatible object storage for medical documents

4. Technology Stack

Layer	Technology	Purpose
Framework	Next.js 14.2	Full-stack React framework with App Router, Server Components, API Routes
Language	TypeScript 5	Type-safe development with strict mode
UI Library	React 18	Component-based UI with hooks and concurrent features
Styling	Tailwind CSS 3.4	Utility-first CSS with custom design tokens
Components	Radix UI	Accessible, unstyled primitives (Dialog, Select, Tabs, Toast, etc.)
State	Zustand 5	Lightweight, immutable global state management
Database	Neon PostgreSQL	Serverless PostgreSQL with pg driver
AI	Google Gemini 3.6 Flash	OCR prescription extraction, medical summarization
Storage	AWS S3	Medical document storage (prescriptions, reports)
Icons	Lucide React	Beautiful, consistent iconography
Forms	Native React	Controlled components with validation
Toasts	Radix Toast	Non-intrusive notification system

5. Main Features — The 6 Portals

5.1 Patient Portal (`PatientPortal.tsx`)

● Purpose: Patient-facing web interface

- **Patient Header Card:** Displays name, Unique Health ID, blood group, gender, age, phone, address
- **Allergies & Chronic Conditions:** Prominently displayed amber/red badges for safety awareness
- **Emergency Contacts:** Quick access to next-of-kin information
- **Lifetime Health Timeline:** Chronological, scrollable list of all medical events (visits, labs, prescriptions, self-reports) with animated reveal on scroll using IntersectionObserver
- **Self-Report Vitals:** Form to submit home readings (blood pressure systolic/diastolic, blood sugar, temperature, weight, symptoms) — persisted to database via `/api/self-report`
- **Health Update Form:** Submit current symptoms and medical history before consultation
- **Medical Document Upload:** Upload prescriptions, lab reports, discharge summaries to S3

5.2 Kiosk Mode (`KioskMode.tsx`)

🎤 Purpose: Walk-in patient registration via voice

- **Voice Input:** Large microphone button with animated pulse rings and wave bars during recording
- **Live Transcription:** Real-time speech-to-text display (simulated with progressive word rendering)
- **Auto-Fill Form:** Patient name, age, complaint, and duration are automatically extracted from voice transcript
- **Multilingual:** Supports English, Hindi, and Bengali voice input and UI
- **Registration Flow:** Creates patient record in DB (or reuses existing by name match) and logs kiosk registration

- **Success Screen:** Confirmation with patient details and option to register another patient

5.3 Lab Portal (`LabPortal.tsx`)

Purpose: Hospital lab test entry and result submission

- **Active Queue:** Searchable patient queue showing name, HID, wait time, and status (Waiting/In Progress/Completed)
- **Test Entry:** Select from predefined test types (CBC, Lipid Panel, Blood Sugar, X-Ray) with parameterized inputs
- **Auto Status Detection:** Automatically determines Normal/High/Low/Critical status based on reference ranges
- **File Upload:** Upload diagnostic files (PDF, JPG, PNG) alongside numeric results
- **Submit to Doctor:** Results are sent to the active doctor session in real-time

5.4 Doctor Clinical View (`DoctorClinical.tsx`)

Purpose: AI-assisted clinical decision support for doctors

- **Patient Selector:** Switch between patients with one click; shows active session badge
- **Chief Complaint:** Editable text area for the patient's primary reason for today's visit
- **AI Clinical Summary:** Real-time analysis by the triage engine (drug interactions, allergy risks, polypharmacy warnings, critical lab alerts, vital sign abnormalities)
- **Health Updates:** Patient-submitted symptoms and medical history
- **Medical Documents:** Uploaded documents associated with the patient
- **Timeline Panel:** Patient's complete medical history in chronological order
- **Lab Results Panel:** Today's lab results with color-coded status indicators
- **Quick Reference:** Allergies and chronic conditions at a glance

5.5 Doctor Prescription Builder (`DoctorPrescription.tsx`)

Purpose: Prescription creation with voice and OCR support

- **Voice Dictation:** Dictate prescriptions naturally (e.g., "Paracetamol 650mg TDS for 5 days") with simulated real-time transcription and auto-populated medication rows
- **OCR Prescription Scanner:** Upload a prescription image → Gemini AI extracts medication names, dosages, frequencies, and durations
- **Manual Entry:** Add/edit/remove medication rows with drug name, dosage, frequency, duration
- **Diagnosis & Clinical Notes:** Free-text fields for diagnosis and clinical observations
- **Save Prescription:** Persists prescriptions to DB while keeping the encounter active
- **Complete Treatment:** Ends the encounter, triggers treatment compaction, creates a summary node

5.6 Patient Intake Wizard (`PatientIntakeWizard.tsx`)

Purpose: Comprehensive lifetime patient registration

- **7-Step Wizard:** Identity & Demographics → Chronic Conditions → Surgical History → Vaccinations → Active Medications → Allergies & Lifestyle → Final Review
- **Unique Health ID:** Auto-generated HID (e.g., HID-8842-X) on registration
- **Phone Verification:** OTP simulation for phone number verification
- **Quick-Add Suggestions:** Common conditions, vaccines, medications, and allergens for fast entry
- **Search & Filter:** Autocomplete search for conditions, vaccines, and medications
- **Step Navigation:** Progress indicator with clickable completed steps
- **Final Review:** Summary of all entered information before submission

6. End-to-End Workflow

Component 1: Initial Patient Input (Point of Entry)

1. **Walk-in Registration:** Patient arrives at hospital kiosk → speaks details in their language → system auto-fills registration form → generates Unique Health ID
2. **Identity Verification:** OTP-based or National ID (Aadhaar) verification audit record is created
3. **Full Registration:** Patient fills 7-step intake wizard (demographics, conditions, surgeries, vaccines, medications, allergies, lifestyle)
4. **Self-Report:** Patient can submit home vitals (BP, sugar, temperature) through the Patient Portal

Component 2: Recurring Input

1. **Hospital Lab:** Lab technician searches patient by name or HID → enters test parameters → uploads diagnostic files → submits results to active doctor session
2. **Doctor Consultation:** Doctor enters patient's HID → reviews complete medical history → AI triage engine analyzes and provides clinical decision support
3. **Prescription:** Doctor dictates or scans prescription → reviews extracted medications → saves to patient record
4. **Document Upload:** MRI, X-Ray, CT Scan reports uploaded from hospital systems

Component 3: Data Storage & Compaction (USP)

1. **Chained Instances:** Each patient interaction (visit, lab, prescription, self-report) creates a "clinical node" chained to the patient's record
2. **Active Treatment:** While treatment is ongoing, all detailed data (vitals, prescriptions, lab results, documents) is preserved in the database
3. **Treatment Completion:** Doctor declares treatment ended → system compacts the treatment chain:
 - Detailed clinical nodes, vitals, prescriptions, lab reports, and documents are **deleted** from the active database
 - A **treatment summary** node is created with diagnosis, notes, key findings, and medication names
 - The encounter status changes from "Active" to "Completed"

4. **Lifetime History:** Compacted summaries remain in the patient's timeline for future reference

Component 4: Patient History Fetching

1. Doctor logs in → enters patient's Unique Health ID
2. System pulls complete medical history: all visits, lab results, prescriptions, treatment summaries
3. Doctor can review history, add prescriptions, or end the current treatment

7. Database Design & Schema

Database Tables (11 tables)

Table	Purpose	Key Fields
patients	Core patient demographics	id, unique_health_id, name, age, gender, blood_group, phone, address, emergency_contacts_json, allergies_json, chronic_conditions_json
encounters	Doctor visit sessions	id, patient_id, doctor_id, date, status (Active/Completed), diagnosis, clinical_notes
clinical_nodes	Timeline events (visits, labs, prescriptions, self-reports)	id, patient_id, encounter_id, node_type, title, description, payload_json
treatment_summaries	Compacted treatment records	id, patient_id, encounter_id, summary, diagnosis, key_findings_json
vitals	Self-reported and clinical vitals	systolic, diastolic, sugar, temperature, weight, symptoms
lab_reports	Lab test reports	test_name, report_date, doctor_id
lab_results	Individual lab test values	test_name, value, unit, reference_range, status
prescriptions	Medication prescriptions	medicine_name, dosage, frequency, duration
identity_verifications	OTP/National ID audit trail	method, phone_last4, national_id_hash
kiosk_registrations	Kiosk walk-in records	name, age, complaint, language, transcript
document_metadata	Uploaded file references	file_name, mime_type, size_bytes, storage_key

health_updates	Patient-submitted symptoms	symptoms, medical_history
----------------	-------------------------------	---------------------------

Schema Features

- **Lazy Initialization:** Schema is created on first API request — no database needed for `next build`
- **Auto-Seeding:** Development mode seeds mock patients, encounters, lab reports, and timeline entries
- **JSONB Columns:** Emergency contacts, allergies, chronic conditions stored as JSONB for flexibility
- **Database Indexes:** Optimized indexes on `patient_id` + `date` for clinical nodes, lab reports, and health updates
- **Transaction Support:** `withTransaction()` helper ensures atomic multi-table operations

8. AI Triage Engine

The triage engine (`triageEngine.ts`) is a rule-based clinical decision support system that runs entirely in-memory (no external API calls). It analyzes the consultation payload and generates:

8 Risk Categories Analyzed

- Allergy Risk Checks:** Direct drug-allergy matching, cross-reactivity detection (e.g., Penicillin → Amoxicillin), severe allergy alerts
- Drug Interaction Checks:** 10 known drug pairs checked (e.g., Warfarin + Aspirin = major bleeding risk)
- Polypharmacy Warning:** Flags when patient has ≥ 5 concurrent medications
- Chronic Condition Monitoring:** Checks for unmedicated conditions, worsening HbA1c, elevated blood pressure
- Vital Signs Assessment:** Hypertensive emergency ($>180/120$), critical SpO2 ($<90\%$), high fever ($>39.5^{\circ}\text{F}$)
- Lab Result Analysis:** 10 critical thresholds for glucose, HbA1c, cholesterol, hemoglobin, creatinine, WBC, platelets, sodium, potassium
- Care Recommendations:** Prioritized actionable items based on findings
- Clinical Summary:** Weighted risk score with severity levels (critical/high/moderate/low/informational)

Drug Allergy Cross-Reactivity Database

Known Allergy	Cross-Reactive Drugs
Penicillin	Amoxicillin, Ampicillin, Piperacillin
Sulfa drugs	Sulfamethoxazole, Sulfasalazine, Celecoxib
Aspirin	Ibuprofen, Naproxen, Diclofenac

💡 Viva Point: Why Rule-Based Instead of ML?

The triage engine uses a deterministic rule-based approach rather than machine learning for several reasons:

- **Explainability:** Every alert has clear evidence and reasoning that can be audited by doctors
- **Reliability:** No risk of model hallucination or unpredictable outputs in critical medical scenarios
- **Compliance:** Deterministic rules are easier to validate for medical device regulations
- **Performance:** Runs in milliseconds with no external API calls

9. Treatment Compaction (Unique Selling Point)

The Core Innovation: After any treatment or procedure is completed, the entire chain of clinical data is compressed. Relevant information for future reference is extracted and stored as a summary node, while the detailed treatment chain is deleted from the active database.

How Treatment Compaction Works

1. **Save Prescription:** Doctor saves prescriptions → data is persisted → encounter remains Active → no data is deleted
2. **Complete Treatment:** Doctor clicks "Declare Treatment Ended & Create Summary"
 - A single PostgreSQL transaction is initiated
 - All prescriptions for the encounter are saved
 - A `treatment_summaries` row is created with diagnosis, notes, and key findings
 - A "Treatment Summary" clinical node is created in the patient's timeline
 - **Detailed data is deleted:** clinical nodes, vitals, lab reports/results, document metadata, prescriptions
 - Encounter status is updated from "Active" to "Completed"
 - Encounter diagnosis and clinical notes are set to NULL (preserved only in summary)

Why This Matters

- **Performance:** Active database stays small — only current treatments have full detail
- **Storage Efficiency:** Detailed data from completed treatments doesn't accumulate indefinitely
- **Historical Preservation:** Treatment summaries contain the essential information for continuity of care
- **Privacy:** Reduces the attack surface by minimizing stored sensitive data

10. OCR & Prescription Extraction

How It Works

1. Doctor uploads a prescription image (JPEG, PNG, WEBP — max 10MB)
2. Image is sent to `/api/ocr` endpoint as FormData
3. Server converts image to base64 and sends to **Google Gemini 3.6 Flash** with a structured prompt
4. Gemini analyzes the image and returns JSON with extracted medications:

```
{
  "rawText": "Full OCR text...",
  "medications": [
    { "medicineName": "Paracetamol", "dosage": "650mg", "frequency": "TDS", "duration": "5"
  ]
}
```

5. Medications are auto-populated in the prescription builder for doctor review
6. Doctor can edit, add, or remove medications before saving

AI Summarization (`/api/summarize`)

Uses Gemini to generate patient-friendly summaries of medical reports. Takes raw medical text and returns a simplified explanation suitable for patients.

11. Multilingual Support

Dual Translation Approach

1. React i18n Context (for Kiosk Mode)

- `I18nProvider` wraps the application with language context
- `useI18n()` hook provides `t(key)` translation function
- Supports English (en), Hindi (hi), Bengali (bn)
- Language persisted in localStorage with key `govehr-language`
- Kiosk-specific translations in `mockData.ts`

2. DOM-Level Translator (for Full Page Translation)

- `DomTranslator` component walks the rendered DOM and replaces English text with Hindi translations
- Uses a TreeWalker to iterate over all text nodes
- Pre-sorted dictionary (longest key first) for greedy substring matching
- Protected from translating: inputs, code blocks, SVG, doctor-facing sections (`data-no-translate` attribute)
- `MutationObserver` integration keeps translations in sync with React re-renders
- Patient data (names, IDs, medical values) are never translated because they don't appear as exact dictionary keys

💡 Viva Point: Why DOM-Level Translation?

The DOM translator was chosen for full-page translation because:

- It translates *everything* without modifying React source code
- Works with dynamically rendered content via `MutationObserver`
- Doctor-facing sections are excluded via `data-no-translate` attribute — ensuring medical terminology stays in English
- Patient data (names, IDs, phone numbers) are automatically excluded since they never match dictionary keys

12. Identity Verification

The `IdentityVerification` component provides two verification methods:

- **OTP Verification:** Simulated OTP sent to phone number → enter code → verification recorded
- **National ID Verification:** Aadhaar-based identity verification with hash storage

Both methods create audit records in the `identity_verifications` table with method, phone last 4 digits (or hashed national ID), and timestamp. The verification state is managed globally via Zustand and persists across portal switches.

13. API Endpoints

Endpoint	Method	Purpose
/api/state	GET	Returns full application state (patients, encounters, labs, timeline, health updates, documents) — used to hydrate the Zustand store on app load
/api/self-report	POST	Saves patient self-reported vitals and creates clinical node
/api/kiosk/register	POST	Registers a walk-in patient via kiosk (creates or reuses patient)
/api/identity/verify	POST	Records identity verification audit (OTP or National ID)
/api/labs	POST	Submits lab report results with associated files
/api/prescriptions/commit	POST	Saves prescriptions while treatment remains active
/api/treatments/complete	POST	Completes treatment and triggers compaction (deletes details, creates summary)
/api/health-updates	POST	Saves patient-submitted symptoms and medical history
/api/documents	POST	Uploads medical documents to S3 and saves metadata
/api/ocr	POST	Extracts medications from prescription image using Gemini AI
/api/summarize	POST	Generates patient-friendly medical report summaries using Gemini AI

14. Setup & Deployment

Prerequisites

- Node.js 18+
- Neon PostgreSQL account (free tier available)
- Google Gemini API key (free tier available)
- AWS S3 bucket (or S3-compatible like Cloudflare R2, MinIO)

Environment Variables

```
DATABASE_URL="postgresql://user:pass@ep-xxx.us-east-2.aws.neon.tech/neondb?sslmode=require"
GEMINI_API_KEY="your-gemini-api-key"
S3_BUCKET="your-bucket-name"
S3_REGION="us-east-1"
S3_ACCESS_KEY_ID="your-access-key"
S3_SECRET_ACCESS_KEY="your-secret-key"
S3_ENDPOINT=""           # Optional: for R2, MinIO
S3_FORCE_PATH_STYLE="false"
S3_PUBLIC_BASE_URL=""     # Optional: for public objects
```

Quick Start

```
npm install
# Set DATABASE_URL and GEMINI_API_KEY in .env.local
npm run dev
```

Available Scripts

Script	Purpose
npm run dev	Start development server (port 3000)
npm run build	Production build with type checking
npm run start	Start production server
npm run lint	Run ESLint checks

15. Viva Preparation — Frequently Asked Questions

Q1: What is the problem your project solves?

Government hospitals in India use fragmented, paper-based systems. Patients visiting multiple hospitals must carry physical files or repeat tests. There is no centralized repository for lifetime health records. Sanjeevni solves this by creating a unified EHR system with a Unique Health ID for every patient, enabling doctors to access complete medical history across all government hospitals.

Q2: What is your Unique Selling Point (USP)?

Treatment Compaction. Unlike traditional EHR systems that accumulate data indefinitely, Sanjeevni compresses completed treatment chains into summary nodes. This keeps the active database lean (only current treatments have full detail) while preserving essential information for continuity of care. It's a novel approach to balancing data completeness with system performance.

Q3: How does the AI Triage Engine work?

It's a deterministic rule-based engine that analyzes 8 categories: drug interactions, allergy cross-reactivity, polypharmacy risks, chronic condition monitoring, vital sign assessment, lab result analysis, and care recommendations. It runs entirely in-memory with no external API calls, ensuring sub-millisecond response times and full explainability for medical audit trails.

Q4: How do you handle multilingual support?

Dual approach: (1) React i18n context for Kiosk Mode with 3 languages (English, Hindi, Bengali), and (2) a DOM-level TreeWalker translator that walks the rendered page and replaces English text with Hindi translations without modifying React source code. Doctor-facing sections are excluded via data-no-translate attributes, and patient data is automatically excluded since it never matches dictionary keys.

Q5: Why PostgreSQL and not MongoDB?

Health records have strong relational integrity requirements — patients have encounters, encounters have prescriptions and lab results, prescriptions belong to specific encounters. PostgreSQL's ACID transactions ensure atomicity for multi-table operations like treatment compaction. JSONB columns provide flexibility for variable-structured data (allergies, emergency contacts) while maintaining relational integrity where needed.

Q6: How does the OCR prescription scanning work?

Doctors upload a prescription image (JPEG/PNG/WEBP). The server sends it to Google Gemini 3.6 Flash (multimodal AI) with a structured extraction prompt. Gemini returns JSON with extracted medications (name, dosage, frequency, duration). The medications are auto-populated in the prescription builder for doctor review and editing before committing to the database.

Q7: How do you ensure data security and privacy?

- Identity verification creates audit trails (OTP/National ID)
- Treatment compaction reduces stored sensitive data after completion
- Server-side API keys never exposed to the client (Gemini key via header, not URL)
- Medical documents stored in S3 with controlled access
- Doctor-facing sections are protected from translation (data-no-translate)
- Phone verification and national ID hashing for identity protection

Q8: How does the system scale?

- **Neon PostgreSQL:** Serverless database that scales automatically with connection pooling (max 10 connections, idle timeout 30s)
- **Treatment Compaction:** Keeps active data volume bounded — completed treatments are compressed to summaries
- **Database Indexes:** Optimized composite indexes on (patient_id, date) for fast timeline queries

- **Lazy Initialization:** Database is created on first request, not at build time
- **S3 Storage:** Medical documents stored in scalable object storage, not the database

Q9: What technologies did you use and why?

- **Next.js 14:** Full-stack framework with API routes, server components, and excellent TypeScript support
- **Zustand:** Lightweight state management (no boilerplate, easy debugging, single store for all 6 portals)
- **Tailwind CSS:** Rapid UI development with consistent design system
- **Radix UI:** Accessible, unstyled primitives (WCAG compliant)
- **Neon PostgreSQL:** Serverless, free tier, compatible with standard pg driver
- **Google Gemini:** Free tier for OCR and summarization, multimodal capabilities

Q10: What would you improve in the future?

- Real Web Speech API integration for actual voice recognition in Kiosk Mode
- DigiLocker API integration for automatic Aadhaar-based document fetching
- End-to-end encryption for patient data in transit and at rest
- Integration with real DICOM viewers for medical imaging (X-Ray, MRI, CT)
- Mobile app (React Native) for patient portal access
- Real-time notifications via WebSocket for lab result delivery
- Role-based access control (RBAC) with hospital-level permissions
- Analytics dashboard for hospital administrators

Q11: Explain the data flow for a typical patient visit.

Flow: Patient registers at kiosk (voice or form) → HID generated → Vitals recorded → Doctor selects patient → AI triage runs → Lab tests ordered → Lab technician enters results → Results appear in doctor view → Doctor creates prescription (voice/OCR/manual) → Treatment completed → Data compacted → Summary node created → Patient can view lifetime timeline in portal.

Q12: How does state management work across 6 portals?

Zustand provides a single global store (`useAppStore`) shared by all 6 portals. The store holds: current role (which portal is active), patient data, encounters, lab reports, timeline, prescriptions, clinical notes, and OCR text. Portal switching is done by changing `currentRole` — no page reload needed. On app load, `loadPersistedState()` fetches data from `/api/state` to hydrate the store from the database.